CS6910 : Fundamentals of Deep Learning

Assignment 2 Report

Team 6

Akash Kumar Paul(CS25S016)
Shrishti Bekal Mudiakal(CS25S009)

Professor: Dr. C. Chandra Sekhar

Department of Computer Science & Engineering
Indian Institute of Technology Madras

Date: 13-04-2026

Contents

1 Task 1: Image Classification using MLFFNN with Deep CNN Features

1.1 Experimental Studies

In this task, pretrained convolutional neural networks are used as fixed feature extractors, and a multi-layer feed-forward neural network (MLFFNN) is trained on top of these extracted features for image classification. All implementations are carried out using the PyTorch framework.

1.1.1 Input Preprocessing and Dataset Handling

- All input images are resized to 224×224.
- Images are converted to tensors using `ToTensor()`.
- The dataset is organized using `ImageFolder`, where each class corresponds to a directory.
- Separate training and test datasets are used as provided, without creating a validation split.

1.1.2 Feature Extraction using Deep CNNs

VGGNet

- A pretrained VGG-16 model is used as the feature extractor.
- The final classification layer is removed, retaining the feature-generating layers.
- The extracted feature vector has a dimensionality of 4096.
- The network parameters are frozen, and no gradients are computed for the CNN.

GoogLeNet

- A pretrained GoogLeNet model is used for feature extraction.
- The final fully connected layer is replaced with an identity mapping.
- The resulting feature vector has a dimensionality of 1024.
- All convolutional layers are frozen during training.

1.1.3 Feature Extraction Procedure

- Images are passed through the pretrained CNNs in batches.
- The output feature maps are flattened into one-dimensional vectors.
- Feature vectors and corresponding labels are stored for both training and test datasets.
- Feature extraction is performed in inference mode to avoid gradient computation.

1.1.4 MLFFNN Classifier Architecture

- The classifier takes CNN feature vectors as input.
- First fully connected layer: 100 neurons followed by ReLU activation.
- Second fully connected layer: 50 neurons followed by ReLU activation.
- Output layer: number of neurons equal to the number of classes.
- The classifier is trained independently while keeping the CNN backbone fixed.

1.1.5 Training Configuration

- Loss function: Cross-Entropy Loss.
- Optimizer: Adam.
- Learning rate: 0.001.
- Number of epochs: 10.
- Batch size: 64 for training the MLFFNN.
- Training is performed on pre-extracted feature vectors rather than raw images.
- Data is shuffled every epoch using random permutation of indices.

1.1.6 Evaluation

- Model performance is evaluated on both training and test feature sets.
- Predictions are obtained by passing feature vectors through the trained MLFFNN.
- Confusion matrices are computed for both training and test datasets.
- Performance comparison is carried out separately for VGGNet-based and GoogLeNet-based feature representations.

1.2 Results

1.2.1 VGGNet

Training Accuracy: 99.94%
Test Accuracy: 94.29%

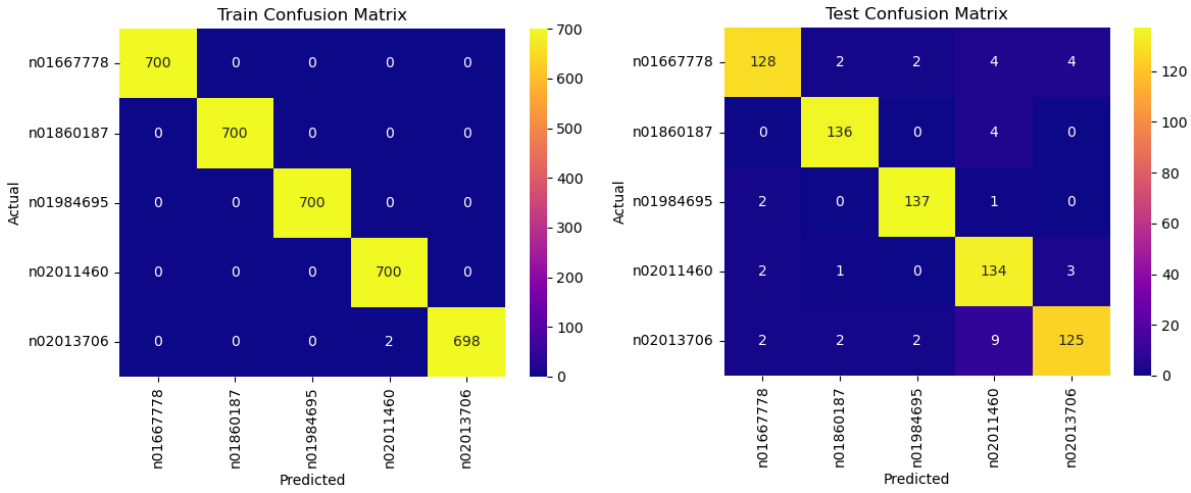


Figure 1: Train and Test Confusion Matrices (VGGNet)

1.2.2 GoogLeNet

Training Accuracy: 99.57%
Test Accuracy: 96.86%

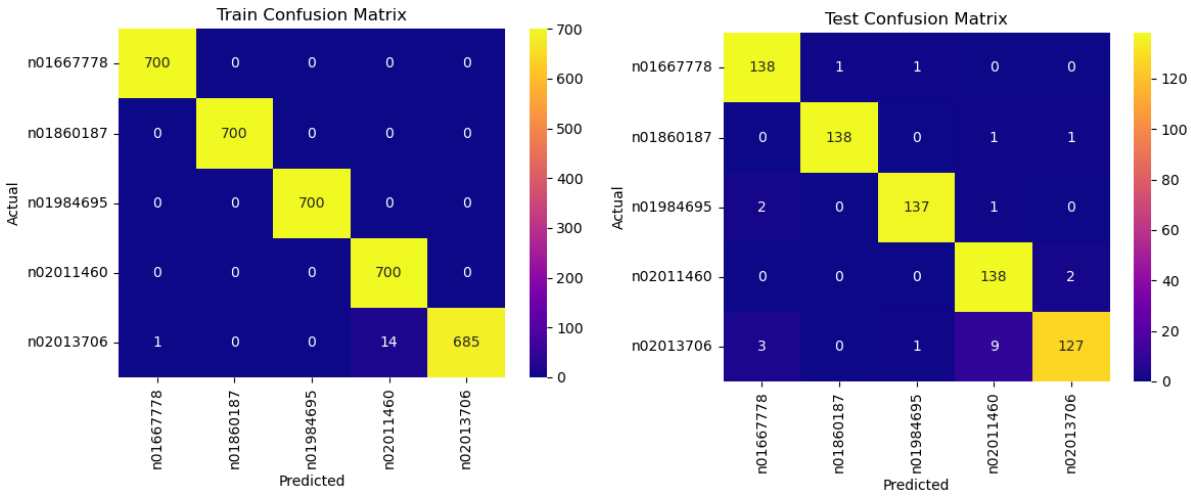


Figure 2: Train and Test Confusion Matrices (GoogLeNet)

1.3 Observations

- Both VGGNet and GoogLeNet achieve very high training accuracy ($> 99\%$), indicating that the MLFFNN classifier is able to effectively learn from the extracted deep features.
- The confusion matrices for both models on the training set are almost perfectly diagonal, showing near-complete memorization of the training data.
- On the test set, GoogLeNet achieves higher accuracy (96.86%) compared to VGGNet (94.29%), suggesting better generalization.
- VGGNet shows slightly more misclassifications across certain classes, particularly in visually similar categories, as seen from off-diagonal entries in the test confusion matrix.
- GoogLeNet produces cleaner and more concentrated confusion matrices, with fewer misclassifications and better class separation.
- The performance gap between training and test accuracy is relatively small for both models, indicating that overfitting is limited.

1.4 Analysis of Results

- Both models achieve very high training accuracy ($\sim 99\%$), confirming that the MLFFNN is able to effectively learn from the extracted CNN features.
- GoogLeNet achieves higher test accuracy (96.86%) compared to VGGNet (94.29%), indicating better generalization on unseen data.
- The confusion matrices show that VGGNet has slightly more off-diagonal entries, meaning more misclassifications across similar classes.
- GoogLeNet produces cleaner predictions with fewer errors, suggesting that its features are more discriminative for this dataset.
- The performance gap between train and test accuracy is small for both models, showing that overfitting is limited due to the use of frozen pretrained features.
- Overall, GoogLeNet provides a better balance between accuracy and generalization in this setup.

2 Task 2: Image Classification using CNN

2.1 Experimental Studies

2.1.1 Input Preprocessing and Dataset Handling

- Images are loaded from separate training and test directories using `ImageFolder`.
- All input images are resized to 224×224.
- Images are converted to tensor format for model input.
- Training data is shuffled, while test data is not shuffled.
- Batch size of 32 is used for both training and testing.

2.1.2 CNN Architecture

- **Convolutional Layer 1 (CL1):**
  - Input channels: 3, Output feature maps: 4
  - Kernel size: 3×3, Stride: 1, Padding: 1
  - Followed by Batch Normalization and ReLU activation
  - Mean pooling with kernel size 2×2, stride 2
- **Convolutional Layer 2 (CL2):**
  - Input channels: 4, Output feature maps: 8
  - Kernel size: 3×3, Stride: 1, Padding: 1
  - Followed by Batch Normalization and ReLU activation
  - Mean pooling with kernel size 2×2, stride 2
- Feature maps are flattened from size $8 \times 56 \times 56$.
- **Fully Connected Layers:**
  - First fully connected layer with 128 neurons
  - ReLU activation applied
  - Dropout with probability 0.5
  - Final output layer maps to number of classes

2.1.3 Training Configuration

- Loss function: Cross-Entropy Loss
- Optimizer: Adam with learning rate 0.001
- Training performed for 10 epochs
- Batch-wise training using forward pass, backpropagation, and parameter updates
- Training and test accuracy computed at each epoch

2.1.4 Evaluation

- Model performance is evaluated on both training and test datasets
- Predictions are obtained using argmax over output logits
- Confusion matrices are computed for both training and test sets
- Heatmaps are used to visualize classification performance across classes

2.2 Results

Training Accuracy: 92.40%
Test Accuracy: 54.57%

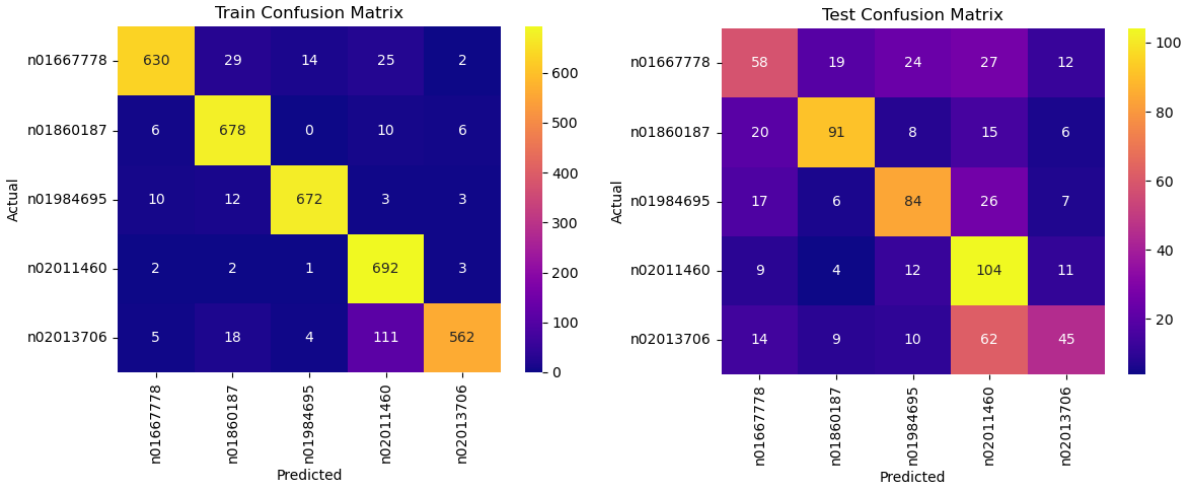


Figure 3: Train and Test Confusion Matrices

2.3 Observations

- The model achieves a high training accuracy of 92.40%, indicating that it is able to learn discriminative features from the training data effectively.
- The test accuracy drops significantly to 54.57%, showing a large generalization gap between training and unseen data.
- The training confusion matrix shows strong diagonal dominance, meaning most samples are correctly classified during training.
- Certain classes (2nd and 4th classes) perform relatively better on the test set, while others (5th class) show significant confusion with neighboring classes.
- The model tends to confuse visually similar classes, as seen from consistent misclassification patterns in the test confusion matrix.

2.4 Analysis of Results

- The large gap between training (92.40%) and test accuracy (54.57%) indicates overfitting, where the model memorizes training data but fails to generalize.

- The small architecture (only 2 convolutional layers with 4 and 8 feature maps) limits the model's ability to capture complex hierarchical features required for robust classification.
- Absence of data augmentation reduces variability in training data, making the model sensitive to small variations in test samples.
- Although batch normalization and dropout are used, they are insufficient to fully regularize the model given the dataset complexity.
- The confusion matrix suggests that feature representations learned are not sufficiently separable for certain classes, leading to overlapping decision boundaries.
- The high-dimensional fully connected layer (flattening $8 \times 56 \times 56$) increases parameter count, which further contributes to overfitting.
- Overall performance is constrained by limited model depth and lack of advanced feature extraction, indicating that deeper architectures or pretrained models would likely give better generalization.

3 Task 3: Sentiment Analysis using RNN

3.1 Experimental Studies

- **Dataset Preparation**
  - Training and test datasets are loaded from CSV files containing tweets and sentiment labels.
  - Text data is tokenized using simple whitespace splitting.
  - Labels are mapped to numerical form: Negative (0) and Positive (1).
- **Vocabulary Construction**
  - A vocabulary is built from the training dataset.
  - Special tokens `<PAD>` and `<UNK>` are included for padding and unknown words.
- **Word Embeddings**
  - Pretrained 200-dimensional GloVe embeddings (glove-wiki-gigaword-200) are used.
  - An embedding matrix is created by mapping vocabulary words to GloVe vectors.
  - Words not found in GloVe are initialized randomly.
  - Embeddings are fine-tuned during training.
- **Input Encoding and Padding**
  - Sentences are converted into sequences of word indices.
  - All sequences are padded to a fixed maximum length.
- **Handling Class Imbalance**
  - Class weights are computed using balanced weighting.
  - These weights are incorporated into the cross-entropy loss function.
- **Model Architecture**
  - A single-layer vanilla RNN is used.
  - Embedding dimension: 200
  - Hidden layer size: 25
  - The final hidden state corresponding to the last valid token is used for classification.
  - A fully connected layer maps hidden representations to output classes.
- **Training Setup**
  - Loss function: CrossEntropyLoss (with class weights)
  - Optimizer: Adam (learning rate = 0.001)
  - Batch size: 32
  - Number of epochs: 10
- **Evaluation**

– Model performance is evaluated on both training and test datasets.
– Predictions are obtained using argmax over output logits.
– Accuracy and classification reports are computed.
– Confusion matrices are generated and visualized using heatmaps.

3.2 Results

Training Accuracy: 100%
Test Accuracy: 92.50%

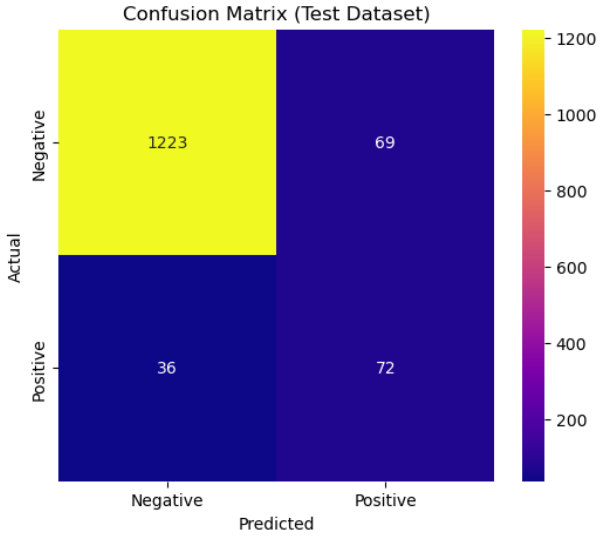


Figure 4: Confusion Matrix (Test Dataset)

Classification Report (Test):

	precision	recall	f1-score	support
Negative	0.97	0.95	0.96	1292
Positive	0.51	0.67	0.58	108
accuracy			0.93	1400
macro avg	0.74	0.81	0.77	1400
weighted avg	0.94	0.93	0.93	1400

Figure 5: Classification Report (Test Dataset)

3.3 Observations

- The model achieves perfect training accuracy, indicating strong fitting on the training data.
- Test accuracy is high (92.50%), demonstrating good generalization performance.
- The model performs very well on the Negative class with high precision (0.97) and recall (0.95).

- The Positive class shows lower precision (0.51) but moderate recall (0.67).
- There is a clear class imbalance, with significantly fewer Positive samples compared to Negative samples.

3.4 Analysis of Results

- The perfect training accuracy suggests overfitting, as the model has memorized the training data.
- Class imbalance impacts the model, resulting in weaker performance on the Positive class despite using class weights.
- Lower precision for the Positive class indicates a higher number of false positives.
- GloVe embeddings help improve overall performance, contributing to strong test accuracy.
- The small hidden size (25) limits model capacity, especially for minority class representation.

4 Task 4: POS Tagging using RNN

4.1 Experimental Studies

- **Embedding: GloVe (200d)**
  - Pre-trained GloVe embeddings (glove-wiki-gigaword-200) are used.
  - Embedding dimension is fixed to 200.
  - An embedding matrix is constructed for the dataset vocabulary.
  - Words not found in GloVe are initialized randomly.
  - Embedding layer is initialized with pretrained weights and fine-tuned during training.
- **Data Loading and Preprocessing**
  - Training and test datasets are loaded from CSV files.
  - Sentences and corresponding POS tags are extracted.
  - Tokenization is performed using whitespace splitting.
  - Vocabulary (word-to-index) and tag-to-index mappings are created.
  - Special tokens `<PAD>` and `<UNK>` are included.
  - Sentences and tags are converted into numerical index sequences.
- **Sequence Preparation**
  - All sequences are padded to a fixed maximum length.
  - Padding is applied using index 0.
  - Both input sentences and tag sequences are padded consistently.
- **Dataset and DataLoader**
  - A custom PyTorch Dataset class is implemented for POS tagging.
  - DataLoader is used for batching and shuffling.
  - Batch size is set to 32.
- **Model Architecture**
  - A single-layer RNN is used for sequence labeling.
  - Input: 200-dimensional word embeddings.
  - Hidden layer size: 25 units.
  - RNN is followed by a fully connected layer to map to tag space.
  - Output is generated for each time step (sequence labeling).
- **Training Setup**
  - Loss function: CrossEntropyLoss with padding ignored.
  - Optimizer: Adam with learning rate 0.001.
  - Number of epochs: 10.
  - Training is performed using mini-batches.
  - Model parameters are initialized with a fixed random seed for reproducibility.

- **Evaluation**
  - Model performance is evaluated on both training and test datasets.
  - Predictions are obtained using argmax over output probabilities.
  - Padding tokens are ignored during evaluation.
  - Accuracy is computed using valid (non-padding) tokens only.
  - Confusion matrices are computed for both training and test sets.
  - Heatmaps are used to visualize classification performance across POS tags.

4.2 Results

Training Accuracy: 99.05%
Test Accuracy: 94.53%

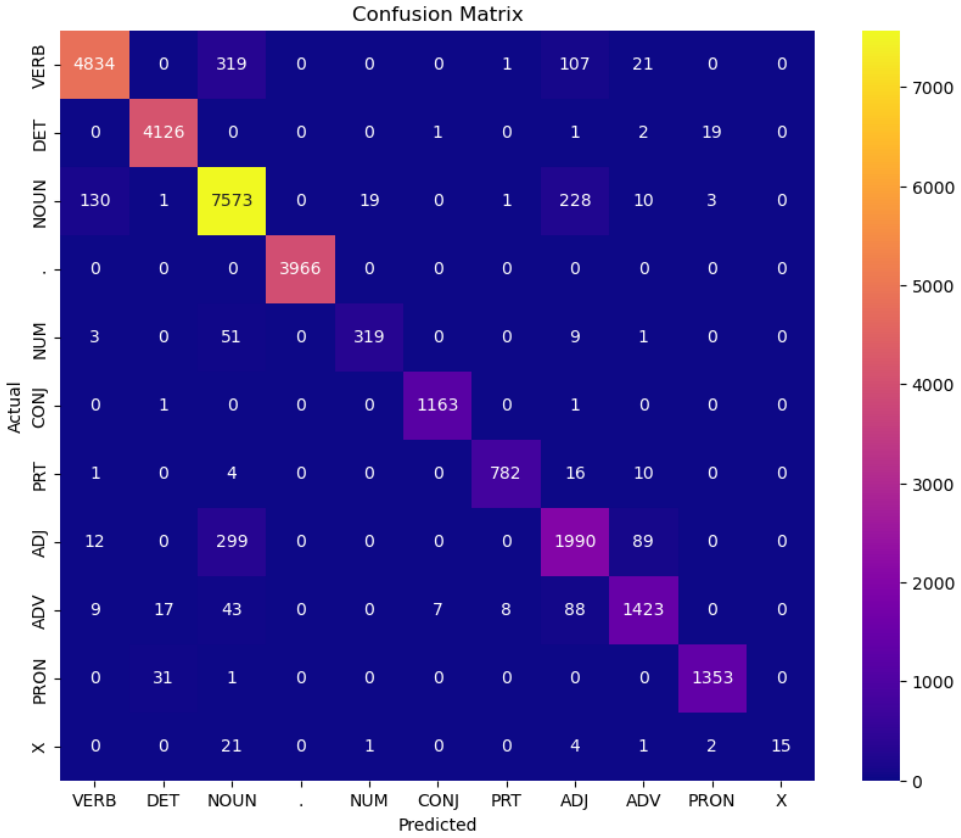
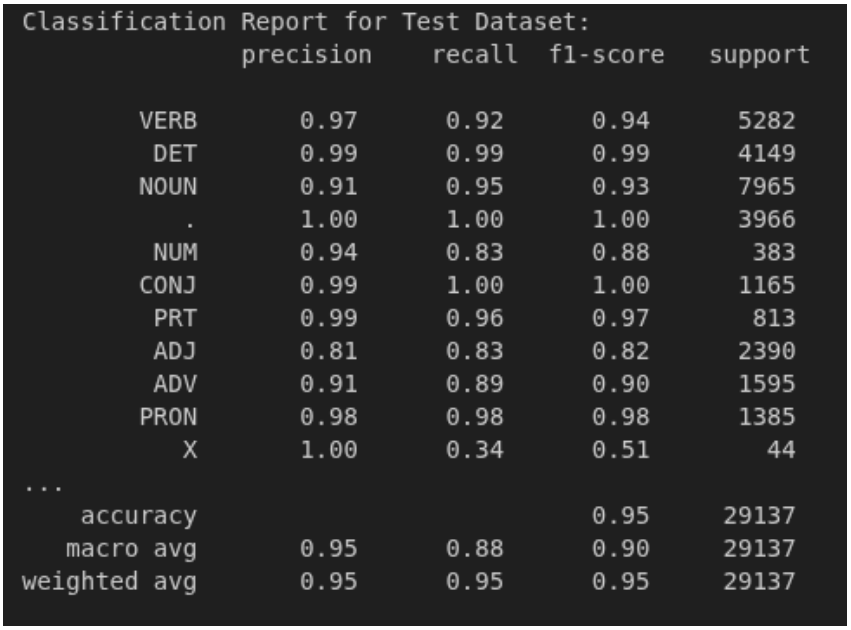


Figure 6: Confusion Matrix for POS Tagging (Test Dataset)



Classification Report for Test Dataset:

	precision	recall	f1-score	support
VERB	0.97	0.92	0.94	5282
DET	0.99	0.99	0.99	4149
NOUN	0.91	0.95	0.93	7965
.	1.00	1.00	1.00	3966
NUM	0.94	0.83	0.88	383
CONJ	0.99	1.00	1.00	1165
PRT	0.99	0.96	0.97	813
ADJ	0.81	0.83	0.82	2390
ADV	0.91	0.89	0.90	1595
PRON	0.98	0.98	0.98	1385
X	1.00	0.34	0.51	44
...				
accuracy			0.95	29137
macro avg	0.95	0.88	0.90	29137
weighted avg	0.95	0.95	0.95	29137

Figure 7: Classification Report (Test Dataset)

4.3 Observations

- The model achieves good test accuracy of 94.53%, indicating good sequence learning.
- Frequent tags such as `DET`, `.`, `CONJ`, and `PRON` are predicted with very high accuracy.
- Confusion is mainly observed between `NOUN` and `VERB`, and between `ADJ` and `ADV`.
- Rare tags like `X` and `NUM` show lower recall due to limited training samples.

4.4 Analysis of Results

- The RNN effectively captures contextual dependencies, improving POS tagging performance.
- Pretrained GloVe embeddings help achieve high accuracy even with a small model.
- Limited hidden size (25 units) restricts the model's ability to distinguish similar tags.
- Class imbalance leads to poorer performance on rare tags.
- Slight gap between train and test accuracy indicates mild overfitting.

******